



TP6 – Migration vers Spring MVC

MVC2 (Servlets) → Spring MVC

Application de Gestion de Produits
avec Authentification et Contrôle d'Accès par Rôle

Étudiante	Zaynab AIT ADDI
Prof	Prof. Mohamed CHERRADI
Année	2025–2026

1. Description du Projet

Ce TP réalise la migration complète d'une application de gestion de produits depuis l'architecture MVC2 (Servlets Java EE) vers Spring MVC 5. Le projet conserve intégralement les couches DAO et métier de TP4/TP5, en remplaçant uniquement la couche web.

L'application permet de gérer un catalogue de produits avec un système d'authentification complet et un contrôle d'accès basé sur les rôles (ADMIN, MANAGER, USER).

2. Objectifs du TP

- Comprendre l'architecture Spring MVC et le rôle du DispatcherServlet (front controller unique)
- Remplacer les Servlets Java EE par des @Controller annotés
- Remplacer les filtres HTTP (javax.servlet.Filter) par un HandlerInterceptor Spring
- Utiliser @Autowired pour l'injection de dépendances (IoC)
- Configurer Spring MVC via XML (spring-mvc-config.xml) et @Configuration Java
- Réutiliser les couches DAO et métier de TP4/TP5 sans modification
- Implémenter un contrôle d'accès granulaire selon le rôle utilisateur

3. Architecture du Projet

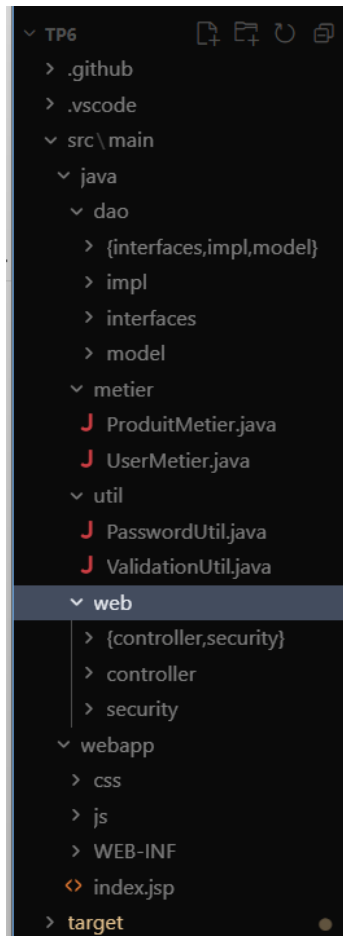
3.1 Structure des fichiers



```

└─ views/
dashboard.jsp,
error/*.jsp
login.jsp, signup.jsp,
addProduit.jsp, editProduit.jsp,

```



3.2 Modèle de données

L'entité Produit possède les attributs suivants :

- idProduit (Long) – identifiant auto-incrémenté
- nom (String, max 100 car.) – nom unique et descriptif
- description (String, max 500 car.) – description complète
- prix (Double) – prix positif en euros
- categorie (String) – Électronique, Accessoires, Logiciels, Livres, Vêtements, Maison, Sports, Autres

L'entité User possède les attributs suivants :

- idUser (Long), email (String), password (String – haché SHA-256)
- nom (String), prenom (String)
- role (enum Role : ADMIN, MANAGER, USER)
- actif (boolean), dateCreation (Date), dateModification (Date)

4. Correspondance MVC2 → Spring MVC

Le tableau suivant montre comment chaque concept MVC2 est remplacé dans Spring MVC :

Concept	MVC2 (TP5)	Spring MVC (TP6)
Point d'entrée HTTP	8 <servlet> dans web.xml	DispatcherServlet unique
Contrôleur	extends HttpServlet	@Controller + @GetMapping/@PostMapping
Paramètre de requête	request.getParameter("x")	@RequestParam String x
Données vers la vue	request.setAttribute("k", v)	model.addAttribute("k", v)
Résolution de vue	dispatcher.forward(req, resp)	return "nom-vue" (ViewResolver)
Redirection	response.sendRedirect("/url")	return "redirect:/url"
Filtre de sécurité	implements Filter	implements HandlerInterceptor
Injection de service	ProduitMetier.getInstance()	@Autowired ProduitMetier metier
Bean DAO	new ProduitDAOImpl() manuel	@Repository (Spring IoC)
Bean Service	Singleton manuel	@Service (Spring IoC)

5. Controllers Spring MVC

5.1 AuthController.java

AuthController remplace 3 Servlets distincts (LoginServlet, SignupServlet, LogoutServlet) en un seul fichier :

Méthode HTTP	URL	Rôle
GET	/login	Affiche la page de connexion (redirige si déjà connecté)
POST	/login	Authentifie l'utilisateur, crée la session (timeout 30 min)
GET	/logout	Invalide la session, redirige vers /login
GET	/signup	Affiche le formulaire d'inscription
POST	/signup	Valide et crée un nouveau compte utilisateur

Points clés du processus de login :

- Validation préalable des champs vides
- Délégation à UserMetier.authenticate() pour la vérification email + mot de passe haché
- Stockage du User complet en session : session.setAttribute("currentUser", user)
- Durée de session : 30 minutes (session.setMaxInactiveInterval(30 * 60))

Points clés du processus de signup :

- Validation email (regex), nom/prénom (min 2 car.), mot de passe fort
- Contrôle de correspondance des deux mots de passe
- Vérification de l'unicité de l'email

- Nouveau compte créé avec rôle USER par défaut

5.2 ProduitController.java

ProduitController regroupe 5 anciens Servlets en un seul fichier :

Ancienne Servlet	Nouvelle méthode	Annotation
DashboardServlet.doGet()	dashboard()	@GetMapping("/dashboard")
AddProduitServlet.doGet()	showAddForm()	@GetMapping("/addProduit")
AddProduitServlet.doPost()	processAdd()	@PostMapping("/addProduit")
EditProduitServlet.doGet()	showEditForm()	@GetMapping("/editProduit")
UpdateProduitServlet.doPost()	processUpdate()	@PostMapping("/updateProduit")
DeleteProduitServlet.doGet()	deleteProduit()	@GetMapping("/deleteProduit")

Fonctionnalités du Dashboard :

- Affichage de la liste complète des produits (vue grille ou tableau)
- Recherche par mot-clé dans le nom, la description et la catégorie
- Calcul des permissions (canAdd, canEdit, canDelete) selon le rôle utilisateur
- Messages de succès/erreur passés en paramètre d'URL (redirect pattern)

Validation dans processAdd / processUpdate :

- Contrôle des champs vides via ValidationUtil.isEmpty()
- Validation du prix (positif, format numérique) via ValidationUtil.isValidPrice()
- Gestion de NumberFormatException

6. Sécurité – AuthenticationInterceptor

6.1 Rôle de l'intercepteur

L'AuthenticationInterceptor remplace l'AuthenticationFilter de MVC2. Il implémente l'interface HandlerInterceptor et son hook preHandle() est exécuté avant chaque requête.

Le flux de traitement est le suivant :

- Si l'URL est publique (login, signup, ressources statiques) → laisser passer
- Sinon, vérifier la présence d'un attribut currentUser en session
- Si non authentifié → rediriger vers /login
- Si authentifié mais rôle insuffisant → HTTP 403 Forbidden
- Sinon → laisser passer la requête

6.2 Contrôle d'accès par rôle

URL	ADMIN	MANAGER	USER
/login, /signup	Public	Public	Public
/logout	✓	✓	✓
/dashboard	✓	✓	✓ (lecture seule)
/addProduit	✓	✓	X (403)
/editProduit	✓	✓	X (403)
/updateProduit	✓	✓	X (403)
/deleteProduit	✓	✓	X (403)

6.3 Enregistrement via WebMvcConfig

La classe WebMvcConfig (@Configuration) enregistre l'intercepteur auprès de Spring MVC, en l'appliquant à tous les chemins (/**) sauf les ressources statiques (/css/**, /js/**, /images/**, /favicon.ico).

7. Configuration Spring MVC

7.1 web.xml – DispatcherServlet

Le web.xml de TP6 est considérablement simplifié par rapport à MVC2. Il ne contient plus qu'un seul servlet (DispatcherServlet) au lieu des 8 précédents, plus un filtre d'encodage UTF-8 Spring.

Élément	MVC2	Spring MVC (TP6)
Nombre de <servlet>	8 Servlets	1 seul DispatcherServlet
Nombre de <servlet-mapping>	8 mappings	1 seul mapping « / »
Filtre d'encodage	Manuel (custom)	CharacterEncodingFilter Spring
Configuration Spring	—	contextConfigLocation → spring-mvc-config.xml
Gestion des sessions	Manuel	session-timeout=30 min, HttpOnly cookie
Pages d'erreur	—	403, 404, 500 → JSP dédiées

7.2 spring-mvc-config.xml

Ce fichier XML configure les quatre éléments essentiels de Spring MVC :

- component-scan : découverte automatique des @Controller, @Service, @Repository dans les packages dao, metier, util, web
- mvc:annotation-driven : active les annotations @RequestMapping, @GetMapping, etc.
- mvc:resources : sert les fichiers CSS, JS et images sans passer par le DispatcherServlet

- `InternalResourceViewResolver` : traduit un nom de vue (ex. "dashboard") en chemin JSP (/WEB-INF/views/dashboard.jsp)

8. Couches DAO et Métier

8.1 Changements dans les DAOs

Les implémentations `ProduitDAOImpl` et `UserDAOImpl` sont identiques à TP5 dans leur logique, avec deux modifications Spring uniquement :

- Ajout de l'annotation `@Repository` – Spring les découvre et gère leur cycle de vie
- Suppression du pattern Singleton manuel – Spring injecte un singleton par défaut

Les données sont stockées en mémoire (`ArrayList`) avec des données initiales :

- 5 produits (Laptop Dell XPS, iPhone 15, Clavier Mécanique, Souris Logitech, Monitor LG UltraWide)
- 3 utilisateurs (admin@example.com, john@example.com, manager@example.com)
- Mots de passe hachés en SHA-256 avec Base64

8.2 Changements dans les Services Métier

`ProduitMetier` et `UserMetier` sont annotés `@Service` et utilisent `@Autowired` pour l'injection de leur DAO :

```
AVANT (MVC2) : ProduitMetier.getInstance() + new ProduitDAOImpl()
APRÈS (Spring): @Service + @Autowired IProduitDAO dao
```

`UserMetier` expose les méthodes : `authenticate()`, `signUp()`, `getUserByEmail()`, `getUserById()`, `getAllUsers()`, `updateUser()`, `deleteUser()`, `changeUserRole()`.

`ProduitMetier` expose les méthodes : `addProduit()`, `deleteProduit()`, `getProduitById()`, `getAllProduits()`, `updateProduit()`, `searchProduits()`.

9. Vues JSP (Interface Utilisateur)

9.1 Pages de l'application

Vue JSP	URL	Description
login.jsp	/login	Formulaire de connexion avec identifiants de test affichés
signup.jsp	/signup	Inscription avec validation des règles de mot de passe
dashboard.jsp	/dashboard	Liste en grille / tableau, recherche, boutons selon rôle
addProduit.jsp	/addProduit	Formulaire d'ajout avec aperçu en direct et compteur de caractères
editProduit.jsp	/editProduit	Formulaire de modification pré-rempli avec aperçu

Vue JSP	URL	Description
error/403.jsp	—	Page d'accès refusé
error/404.jsp	—	Page introuvable
error/500.jsp	—	Erreur interne du serveur

Connexion
Gestion des Produits

Adresse Email
admin@example.com

Mot de passe

☐ Se souvenir de moi

Se Connecter

Identifiants de Test

Admin: admin@example.com / admin123
User: john@example.com / password123
Manager: manager@example.com / manager123

Pas encore de compte? [S'inscrire ici](#)

Capture : Page de connexion de l'application

Inscription
Créez votre compte

Prénom *
Ex: Jean

Nom *
Ex: Dupont

Adresse Email *
exemple@domaine.com

Mot de passe *
Minimum 8 caractères

Requis:

- Minimum 8 caractères
- Au moins 1 majuscule (A-Z)
- Au moins 1 minuscule (a-z)
- Au moins 1 chiffre (0-9)

Confirmer le mot de passe *
Confirmez votre mot de passe

☐ J'accepte les [conditions d'utilisation](#)

S'inscrire

Déjà inscrit? [Se connecter ici](#)

Capture : Page d'inscription (Sign up)

Gestion Produits Accueil Ajouter Produit Principal Admin Administrateur Déconnexion

Gestion des Produits

Bienvenue, Principal! Voici la liste complète de vos produits.

Rechercher un produit... Rechercher

Tableau Grille

#1
Laptop Dell XPS
Électronique
Ordinateur portable high-performance
1299.99 €
Modifier Supprimer

#2
iPhone 15
Électronique
Smartphone dernier modèle
999.99 €
Modifier Supprimer

#3
Clavier Mécanique
Accessoires
Clavier gaming RGB
149.99 €
Modifier Supprimer

#4
Souris Logitech
Accessoires
Souris sans fil haute précision
49.99 €
Modifier Supprimer

#5
Monitor LG UltraWide
Électronique
Écran 34 pouces ultrawide
549.99 €
Modifier Supprimer

© 2026 Gestion des Produits - JEE MVC1 | Tous droits réservés

Capture : Tableau de bord – Liste des produits

Gestion Produits Accueil Ajouter Produit Principal Admin Administrateur Déconnexion

Gestion des Produits

Bienvenue, Principal! Voici la liste complète de vos produits.

Rechercher un produit... Rechercher

Tableau Grille

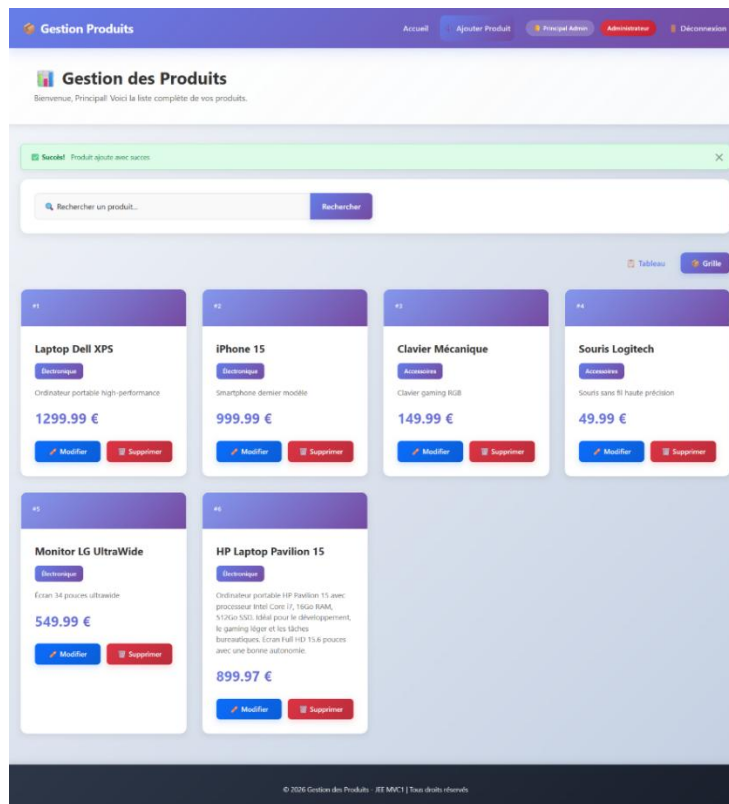
ID	Nom	Description	Catégorie	Prix	Actions
#1	Laptop Dell XPS	Ordinateur portable high-performance	Électronique	1299.99 €	Modifier Supprimer
#2	iPhone 15	Smartphone dernier modèle	Électronique	999.99 €	Modifier Supprimer
#3	Clavier Mécanique	Clavier gaming RGB	Accessoires	149.99 €	Modifier Supprimer
#4	Souris Logitech	Souris sans fil haute précision	Accessoires	49.99 €	Modifier Supprimer
#5	Monitor LG UltraWide	Écran 34 pouces ultrawide	Électronique	549.99 €	Modifier Supprimer

© 2026 Gestion des Produits - JEE MVC1 | Tous droits réservés

Capture : Dashboard – Vue tableau des produits

The screenshot shows the 'Ajouter un Nouveau Produit' form. At the top, there's a green button 'Ajouter un Nouveau Produit' and a link 'Retour à la liste des produits'. Below is a green bar with 'Remplissez le formulaire ci-dessous' and 'Validation en cours'. The form itself is titled 'Formulaire d'ajout' and contains several fields: 'Nom du produit' (with 'HP Laptop Pavilion 15' entered), 'Description' (with a detailed description of the laptop), 'Prix (€)' (with '899.99' entered), and 'Catégorie' (with 'Electronique' selected). There's also a 'Description complète du produit' field. At the bottom, there are two buttons: 'Ajouter le produit' and 'Annuler'.

Capture : Addition d'un produit



Capture : Après l'addition

Gestion Produits Accueil Principal Admin Déconnexion

Modifier un Produit

Mettez à jour les informations du produit dans le système

[← Retour à la liste des produits](#)

PRODUIT #1
Laptop Dell XPS Modification en cours

Formulaire de modification

Nom du produit *
Laptop Dell XPS
Le nom doit être unique et descriptif

Description *
Ordinateur portable high-performance
Description complète du produit (modèle, dimensions, fonctionnalités...)

Prix (€) * **Catégorie ***
1299.99 Prix en euros (TTC) Electronique Catégorie pour organiser les produits

Aperçu :
Nom: Laptop Dell XPS **Prix: 1299.99 €**

Mettre à jour Annuler

Capture : Modifier un produit

Gestion Produits Accueil Ajouter Produit Principal Admin Administrateur Déconnexion

Gestion des Produits

Bienvenue, Principal! Voici la liste complète de vos produits.

Rechercher X Réinitialiser

Tableau Grille

#2

iPhone 15

Electronique

Smartphone dernier modèle

999.99 €

Modifier Supprimer

© 2025 Gestion des Produits - JEE MVC1 | Tous droits réservés

Capture : La recherche

9.2 Technologies et librairies front-end

- Bootstrap 5.3 (CDN) – mise en page responsive, navbar, cartes, tableaux, alertes
- Font Awesome 6.4 (CDN) – icônes dans les boutons et labels
- JSTL Core (<c:forEach>, <c:if>, <c:choose>) – itération et conditions côté serveur
- CSS custom – thème dégradé violet/bleu (linear-gradient #667eea → #764ba2)
- JavaScript vanilla – aperçu en direct du formulaire, toggle grille/tableau, validation client

9.3 Fonctionnalités clés du Dashboard

- Vue double : grille de cartes (défaut) et tableau HTML (basculable par boutons)
- Barre de recherche GET avec réinitialisation
- Message d'information pour les utilisateurs USER (mode consultation)
- Boutons Modifier et Supprimer masqués / remplacés par icône verrou pour USER
- Confirmation JavaScript avant suppression d'un produit

10. Dépendances Maven (pom.xml)

GroupId	ArtifactId	Version	Scope
org.springframework	spring-webmvc	5.3.30	compile
org.springframework	spring-context	5.3.30	compile
javax.servlet	javax.servlet-api	4.0.1	provided
javax.servlet.jsp	javax.servlet.jsp-api	2.3.3	provided
javax.servlet	jstl	1.2	compile
org.apache.taglibs	taglibs-standard-impl	1.2.5	compile
org.slf4j	slf4j-api + slf4j-simple	1.7.36	compile
junit	junit	4.13.2	test

Le build génère un fichier WAR nommé gestion-produits-spring.war grâce au plugin maven-war-plugin 3.4.0. Un plugin tomcat7-maven-plugin est également configuré pour le déploiement local rapide sur le port 8080.

11. Installation et Déploiement

11.1 Prérequis

Outil	Version requise
Java JDK	11
Apache Maven	3.8.x ou supérieur
Apache Tomcat	9.x
Spring MVC	5.3.30
Navigateur	Tout navigateur moderne (Chrome, Firefox, Edge)

```

PS C:\Users\Zayna\Desktop\S4\JEE\TPs JEE\TP6> # Java setup
PS C:\Users\Zayna\Desktop\S4\JEE\TPs JEE\TP6> $env:JAVA_HOME="C:\Program Files\Java\jdk-11"
PS C:\Users\Zayna\Desktop\S4\JEE\TPs JEE\TP6> $env:Path="$env:JAVA_HOME\bin;$env:Path"
  
```

11.2 Étapes de déploiement

- **Compiler le projet :** `mvn clean package`
- **Déployer le WAR :** copier `target/gestion-produits-spring.war` dans le dossier webapps de Tomcat
- **Démarrer Tomcat et accéder à :** `http://localhost:8080/TP6/login`

```

PS C:\Users\Zayna\Desktop\S4\JEE\TPs JEE\TP6> # Tomcat
PS C:\Users\Zayna\Desktop\S4\JEE\TPs JEE\TP6> $TOMCAT_HOME="C:\Tomcat"
PS C:\Users\Zayna\Desktop\S4\JEE\TPs JEE\TP6> $env:CATALINA_HOME=$TOMCAT_HOME
PS C:\Users\Zayna\Desktop\S4\JEE\TPs JEE\TP6> $env:CATALINA_BASE=$TOMCAT_HOME
PS C:\Users\Zayna\Desktop\S4\JEE\TPs JEE\TP6>
PS C:\Users\Zayna\Desktop\S4\JEE\TPs JEE\TP6> # 🔥 Go to ROOT project
PS C:\Users\Zayna\Desktop\S4\JEE\TPs JEE\TP6> Set-Location "C:\Users\Zayna\Desktop\S4\JEE\TPs JEE\TP6"
PS C:\Users\Zayna\Desktop\S4\JEE\TPs JEE\TP6>
PS C:\Users\Zayna\Desktop\S4\JEE\TPs JEE\TP6> # Clean build
PS C:\Users\Zayna\Desktop\S4\JEE\TPs JEE\TP6> Remove-Item -Path ".\target" -Recurse -Force -ErrorAction SilentlyContinue
PS C:\Users\Zayna\Desktop\S4\JEE\TPs JEE\TP6> mvn clean package
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.gestionproduits:gestion-produits-spring >-----
[INFO] Building Gestion des Produits - Spring MVC 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ war ]-----
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ gestion-produits-spring ---
[INFO]
[INFO] --- resources:3.4.0:resources (default-resources) @ gestion-produits-spring ---
[INFO] skip non existing resourceDirectory C:\Users\Zayna\Desktop\S4\JEE\TPs JEE\TP6\src\main\resources
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ gestion-produits-spring ---
[INFO] Changes detected - recompiling the module! :source
[INFO] Compiling 15 source files with javac [debug target 11] to target\classes

```

Cursor-Tab → Zaynab AIT ADDI (1 week ago)

11.3 Comptes de test

Email	Mot de passe	Rôle	Permissions
admin@example.com	admin123	ADMIN	Toutes opérations
manager@example.com	manager123	MANAGER	Voir + CRUD produits
john@example.com	password123	USER	Voir seulement

12. Problèmes Courants et Solutions

#	Problème	Cause	Solution
1	Bean non trouvé au démarrage	Classe non annotée @Service ou @Repository	Vérifier les annotations et le base-package du component-scan
2	ViewResolver ne trouve pas les JSP	Préfixe ou suffixe mal configuré	Configurer /WEB-INF/views/ comme préfixe et .jsp comme suffixe
3	Boucle infinie de redirection	/login intercepté par l'intercepteur	Exclure /login et /signup dans isPublicURL()
4	Caractères français corrompus	CharacterEncodingFilter manquant	Filtre UTF-8 Spring dans web.xml avec forceEncoding=true
5	404 sur CSS/JS	DispatcherServlet intercepte les ressources	Configurer mvc:resources dans spring-mvc-config.xml

#	Problème	Cause	Solution
6	HTTP 403 inattendu	Logique de rôle incomplète dans l'intercepteur	Vérifier hasPermission() dans AuthenticationInterceptor

13. Notes et Perspectives

13.1 Limitations actuelles

- Les données sont stockées en mémoire (ArrayList) – volatiles au redémarrage du serveur
- Pas de pagination sur la liste des produits
- La recherche porte sur tous les produits, sans filtrage par catégorie

13.2 Évolutions vers Spring Boot

Ce TP constitue la base pour les évolutions suivantes :

- Spring Boot – auto-configuration, suppression de web.xml et spring-mvc-config.xml
- Spring Data JPA – persistance en base de données (MySQL, PostgreSQL)
- Spring Security – remplacement de l'intercepteur par un framework de sécurité complet
- Thymeleaf – moteur de templates moderne en remplacement de JSP